

Day 3 Lab Sheet: Build the Notes Board Server with Express

Goal: By the end of this lab your Notes Board keeps its notes on a **server** that you wrote with Express. Reading notes is public. Adding and deleting notes needs a **password**, which is kept in a private `.env` file.

Time: about 100 minutes, plus the quiz.

What you need: Your `Workshop` folder with the Day 2 result (`notes-board` and `hello-server`). VS Code with a **Command Prompt** terminal. Node.js 24 from Day 2.

If your Day 2 files are missing or broken, copy the **Day 3 starter** folder from your instructor. It has everything finished up to Day 2. The **done** folder is the finished result for today.

Remember the Day 2 habit: after every part, do the **Check** line. If the Check is not true, stop and fix it before you go on. Raise your hand if you are stuck.

Important: Our server runs in the terminal and keeps the terminal busy. After you change a server file (`server.js` , `data.js` or `.env`), you must **stop** the server with **Ctrl + C** and **start** it again with `node server.js` . A running server does not reload by itself.

Part 0: Check your setup and your starter (3 minutes)

1. Open VS Code and open your `Workshop` folder.
2. Open a **Command Prompt** terminal (Terminal, then New Terminal, then the down arrow next to **+**, then **Command Prompt**).
3. Type each command and press **Enter**:

```
node -v
```

```
node setup-check.js
```

4. In the VS Code file list, open the `notes-board` folder. You should see `index.html` , `about.html` , a `css` folder and a `js` folder.

Check: `node -v` starts with `v24` . `node setup-check.js` shows four **PASS** lines. Your `notes-board` folder has the files named above.

If the setup check says **FAIL** for port 3000, an old server (for example `hello.js` from Day 2) is still running in another terminal. Find it and press **Ctrl + C**.

Part 1: Make a `public` folder (7 minutes)

Our server will send only the files in a folder named `public`. This keeps our private files, such as `server.js` and `.env`, out of reach of visitors.

1. In the VS Code file list, right-click the `notes-board` folder and choose **New Folder**. Name it `public`.
2. **Drag and drop** these four items from `notes-board` **into** the `public` folder:
 - `index.html`
 - `about.html`
 - the `css` folder
 - the `js` folder
3. If VS Code asks "Are you sure you want to move...?", click **Move**.

After this, your folder must look like this:

```
Workshop
  hello-server
  notes-board
    public
      about.html
      index.html
      css
        style.css
      js
        app.js
        about.js
        student-details.js
```

Check: Directly inside `notes-board` there is only the `public` folder (for now). Inside `public` there are `index.html`, `about.html`, `css` and `js`.

Part 2: Start a Node project and install Express (10 minutes)

All commands from now on run **inside** `notes-board`.

1. In the terminal, go into the folder:

```
cd notes-board
```

2. Check that you are in the right place:

```
dir
```

You should see `public` in the list.

3. Make the `package.json` file. The `-y` means "answer yes to every question":

```
npm init -y
```

4. Open `package.json` in VS Code and read it. It has the name of your project and a `scripts` part.

5. Install Express. **You need the internet for this step.**

```
npm install express
```

6. Wait until the prompt comes back. Look at the VS Code file list. New things appeared: a `node_modules` folder and a `package-lock.json` file. Open `package.json` again. It now has a `dependencies` part with `express`.

7. In `package.json`, change **two** things.

Change the line `"main": "index.js",` to:

```
"main": "server.js",
```

Replace the whole `"scripts": { ... },` block with:

```
"scripts": {  
  "start": "node server.js"  
},
```

8. Save the file (**Ctrl + S**).

If the internet is slow or fails, your instructor may give you a zipped `node_modules` folder. Follow the instructor's steps. Do not worry. It is the same Express code.

The Express version that this lab was tested with is **5.2.1**. Your number may be a little newer, and that is fine.

Check: Type this and press Enter. It must show `express@5` followed by a version number:

```
npm ls express
```

If you do not see `express`, see the Stuck Sheet.

Part 3: Your first Express server (10 minutes)

Now we write the server. We build `server.js` in small steps during the lab. At the end it will be the same as the finished file.

1. Click the `notes-board` folder (**not** `public`). Click **New File** and name it `server.js`.
2. Type this code:

File: `notes-board\server.js`

```
// server.js (Day 3)
//
// The Notes Board server, written with Express.
// It does two jobs:
// 1. It sends our web pages (the files in the "public" folder).
// 2. It offers an API: a way for JavaScript in the browser to
//    read, add and delete notes.
//
// Run it:  node server.js
// Open it: http://localhost:3000

const path = require("node:path");
const express = require("express");

const PORT = process.env.PORT || 3000;

const app = express();

// Send the web pages.
app.use(express.static(path.join(__dirname, "public")));

app.listen(PORT, function (error) {
  if (error) {
    console.error("Could not start the server: " + error.message);
    console.error("Is another server already using port " + PORT + "?");
    process.exit(1);
  }
  console.log("Notes Board is running. Open http://localhost:" + PORT);
});
```

What the code does:

- `require("express")` loads the package we installed.
- `express()` makes our server, and we call it `app`.
- `express.static(...)` is **middleware**. It sends the files of the `public` folder.
- `__dirname` is the folder where `server.js` is. `path.join` joins folder names safely.
- `app.listen(PORT, ...)` starts the server. `PORT` is 3000, unless a setting says another number.

3. Save, and start the server in the terminal:

```
node server.js
```

4. The terminal prints `Notes Board is running...` and **stays busy**. This is correct.
5. Open your browser and go to `http://localhost:3000`. You see your Notes Board from Day 2. Click **About**. It works too.
6. Now try `http://localhost:3000/nothing`. The page says `Cannot GET /nothing`. This is Express saying "no file or route is at this address" (status 404).
7. Go to the terminal and press **Ctrl + C** to stop the server.

Check: Your Notes Board opens at `http://localhost:3000`, and you can explain: "The browser sent a request. Express found the file in `public` and sent it back."

Today we open the page with `http://localhost:3000`. Do **not** double-click `index.html` any more. The new JavaScript needs the server.

Part 4: The notes live in `data.js` (10 minutes)

We put the notes in their own file. `server.js` will ask this file for notes. On Day 4 we will replace the inside of these functions with a database, and the rest of the app will stay the same.

1. Click the `notes-board` folder (**not** `public`), click **New File**, and name it `data.js`.
2. Type the code below, **one section at a time**. Your instructor will explain each section before you type it.

Section 1: The notes

File: `notes-board\data.js`

```
// data.js (Day 3)
//
// This file keeps the notes in memory (a JavaScript array).
// When the server stops, the notes are gone. On Day 4 we replace
// the inside of these three functions with a real database.
// The names stay the same, so server.js does not need to change much.

const notes = [
  { id: 1, title: "Welcome", text: "This note came from the server." },
  { id: 2, title: "Study plan", text: "Learn HTML on Monday, CSS on Tuesday, and
JavaScript on Wednesday." }
];

let nextId = 3;
```

Section 2: Read all notes

Add this below Section 1:

```
// Give back all notes, newest first.  
async function listNotes() {  
  return [...notes].sort(function (a, b) {  
    return b.id - a.id;  
  });  
}
```

- `async` marks a function that gives back a **promise**. We write it now because a database needs waiting on Day 4.
- `[...notes]` makes a copy of the array, so that sorting does not change the original.
- `b.id - a.id` puts the note with the highest `id` first.

Section 3: Add a note

Add this below Section 2:

```
// Save a new note and give it back (with its new id).  
async function addNote(title, text) {  
  const note = { id: nextId, title: title, text: text };  
  nextId = nextId + 1;  
  notes.push(note);  
  return note;  
}
```

Section 4: Delete a note, and share the three functions

Add this below Section 3:

```
// Remove a note. Gives back true if it existed, false if it did not.
async function deleteNote(id) {
  const index = notes.findIndex(function (note) {
    return note.id === id;
  });

  if (index === -1) {
    return false;
  }

  notes.splice(index, 1);
  return true;
}

module.exports = { listNotes, addNote, deleteNote };
```

`module.exports` shares the three functions, so that `server.js` can use them.

3. Save the file.

Check: `data.js` is in `notes-board` (next to `server.js`), not inside `public`. Nothing runs yet. We use it in the next part.

Part 5: The first API route: GET (10 minutes)

1. Open `server.js`. **Below** the line `const express = require("express");` add:

```
const { listNotes, addNote, deleteNote } = require("./data");
```

2. **Below** the `express.static` block and its blank line, and **above** `app.listen(...)`, add:

```
// API 1: read all notes (public, no password).
app.get("/api/notes", async function (request, response) {
  const notes = await listNotes();
  response.json(notes);
});
```

(Leave one blank line after the block, before `app.listen`.)

3. Save. **Restart the server:** click the terminal, press **Ctrl + C**, then type `node server.js` again. (Tip: the **Up arrow** brings back your last command.)

4. In the browser, open `http://localhost:3000/api/notes` .

You see your notes as **JSON text**: a list with two notes. The newest note is first. This is not a page. It is **data**. This is our API.

5. Now look at it in the browser tools:

- Press **F12** and click the **Network** tab.
- Reload the page with **F5**.
- Click the request named `notes` .
- Find the **status code** (it is 200) and the **method** (it is GET).
- Find the **Response** (or **Preview**) tab. It shows the JSON.

6. Close the tools with **F12**.

Check: You can read the two notes as JSON at `/api/notes` , and you saw status **200** in the Network tab. Your old page at `http://localhost:3000` still works the Day 2 way. We change it in Part 8.

Part 6: The password, `.env` and `.gitignore` (12 minutes)

The password must not be written in `server.js` . We keep it in a private file.

6a. Create `.env.example`

1. Click the `notes-board` folder, click **New File**, and name it `.env.example` (it starts with a dot).
2. Type:

File: `notes-board\.env.example`

```
# Copy this file to a new file named .env and change the value.
# The .env file is private. Never share it and never upload it to GitHub.
NOTES_PASSWORD=change-this-password
PORT=3000
```

This is a **sample**. It is safe to share, because it has a fake password.

6b. Create your private `.env`

1. In the terminal, make a copy:

```
copy .env.example .env
```

2. Open the new file `.env` in VS Code. Change the password to **your own**. Use a word you will remember. **Do not use a password that you use anywhere else.**

File: `notes-board\.env` (example, write your own password)

```
NOTES_PASSWORD=choose-your-own-password
PORT=3000
```

3. Save.

Warning: Make this file only in VS Code or with the `copy` command. If you make it in Notepad, Windows may save it as `.env.txt`, and the server will not find it. Check the name in the VS Code file list. It must be exactly `.env`.

6c. Create `.gitignore`

Create a new file `.gitignore` in `notes-board` and type:

File: `notes-board\.gitignore`

```
# Packages (anyone can get them again with: npm install)
node_modules/

# Private settings (passwords live here)
.env

# Database files (added on Day 4)
*.sqlite
*.sqlite3
*.db
```

We are not using Git today. We write this file now, so that on Day 4 the password and the packages are never uploaded.

6d. Make the server read `.env`

Open `server.js`.

1. Find the line `const PORT = process.env.PORT || 3000;`. **Above** it (and below the blank line that follows the `require` lines), add:

```
// Read the private settings from the .env file, if there is one.
// (On a hosting service the settings come from the service itself.)
try {
  process.loadEnvFile();
} catch (error) {
  console.log("No .env file found. Using the settings of this computer.");
}

const PASSWORD = process.env.NOTES_PASSWORD;
```

2. **Below** the `PORT` line and its blank line, and above `const app = express();`, add:

```
// Stop early with a clear message if the password is missing.
if (!PASSWORD) {
  console.error("NOTES_PASSWORD is not set.");
  console.error("Copy .env.example to .env and write a password in it.");
  process.exit(1);
}
```

(Leave one blank line after the block.)

3. Save. Restart the server (**Ctrl + C**, then `node server.js`).

6e. See the safety check work (2 minutes)

1. Stop the server with **Ctrl + C**.
2. In VS Code, right-click `.env` and choose **Rename**. Change the name to `env-off`.
3. Run `node server.js`. The server does **not** start. It prints `NOTES_PASSWORD is not set.` and the prompt comes back.
4. Rename the file back to `.env`. Run `node server.js` again. It starts.

Check: The server starts with your `.env` in place, and it refuses to start when `.env` is missing. Your password appears **nowhere** in `server.js`.

Part 7: POST and DELETE with `requirePassword` (12 minutes)

Now we add the two routes that change data, and the middleware that guards them. Open `server.js`.

1. **Above** the line `// Send the web pages.` add (with the blank line after it):

```
// Let the server read JSON sent by the browser.  
app.use(express.json());
```

2. **Above** the line `// API 1: read all notes...` add (with the blank line after it):

```
// A small helper: only let a request continue if the password is right.  
function requirePassword(request, response, next) {  
  if (request.get("x-notes-password") !== PASSWORD) {  
    response.status(401).json({ error: "Wrong password." });  
    return;  
  }  
  next();  
}
```

3. **Below** the GET route and its blank line, and **above** `app.listen(...)`, add:

```
// API 2: add a note (password needed).
app.post("/api/notes", requirePassword, async function (request, response) {
  const title = String(request.body.title || "").trim();
  const text = String(request.body.text || "").trim();

  if (title === "" || text === "") {
    response.status(400).json({ error: "Please write both a title and a note." });
    return;
  }
  if (title.length > 60 || text.length > 300) {
    response.status(400).json({ error: "The title or the note is too long." });
    return;
  }

  const note = await addNote(title, text);
  response.status(201).json(note);
});

// API 3: delete a note (password needed).
app.delete("/api/notes/:id", requirePassword, async function (request, response) {
  const id = Number(request.params.id);
  const found = await deleteNote(id);

  if (!found) {
    response.status(404).json({ error: "That note does not exist." });
    return;
  }
  response.status(204).end();
});
```

(Leave one blank line after the block, before `app.listen`.)

Notice:

- `requirePassword` is written **between** the route address and the route function. Only these two routes are guarded.
- The password comes from the request **header** `x-notes-password`.
- The server answers **201** when a note is created, **204** when a note is deleted, **400** for a bad note, **401** for a wrong password, and **404** for a note that is not there.

4. Save. Restart the server (**Ctrl + C**, then `node server.js`).

7a. Test the routes from the browser Console

A browser bar can only send GET. To send POST and DELETE, we use JavaScript.

1. Open `http://localhost:3000` in the browser. Press **F12** and click the **Console** tab.
2. Type this and press **Enter**. It makes a small helper that tries to add a note:

```
async function tryAdd(password, title, text) {
  const response = await fetch("/api/notes", {
    method: "POST",
    headers: { "Content-Type": "application/json", "x-notes-password": password },
    body: JSON.stringify({ title: title, text: text })
  });
  console.log(response.status, await response.json());
}
```

Some browsers refuse to paste code into the Console and ask you to type `allow pasting` first. If you see that, type it, press Enter, and paste again. Or type the code by hand.

3. Try a **wrong password**:

```
tryAdd("wrong", "Test", "Hello")
```

It prints `401` and `{error: "Wrong password."}`.

4. Try the **right password** (write your own password from `.env` inside the quotes), with an **empty title**:

```
tryAdd("your-password-here", "", "Hello")
```

It prints `400` and an error text.

5. Now add a real note with the right password:

```
tryAdd("your-password-here", "From the Console", "Sent without the form")
```

It prints `201` and the new note with its `id`.

6. Open `http://localhost:3000/api/notes` in a new tab. Your new note is in the list.

7. Make a helper for delete and try it. Use the `id` of the note you made:

```

async function tryDelete(password, id) {
  const response = await fetch("/api/notes/" + id, {
    method: "DELETE",
    headers: { "x-notes-password": password }
  });
  console.log(response.status);
}

```

```
tryDelete("wrong", 3)
```

This prints `401` .

```
tryDelete("your-password-here", 999)
```

This prints `404` , because note 999 does not exist.

```
tryDelete("your-password-here", 3)
```

This prints `204` (use the real `id` of your test note instead of 3).

Check: You saw **401** for the wrong password, **400** for the empty title, **201** for the added note, **404** for a missing note and **204** for a deleted note. This shows that the **server** protects the password, not the page.

Close the tools with **F12**.

Part 8: Connect the page to the server (17 minutes)

The page still uses its Day 2 JavaScript. We now change it to talk to the server.

8a. Add a password box to `public\index.html`

Open `public\index.html` . Find the title input line (`<input type="text" id="note-title" ...>`).

Below it, and **above** `<label for="note-text">` , add (with a blank line after it):

```

<label for="note-password">Password</label>
<input type="password" id="note-password" name="password" placeholder="Needed
to add or delete notes" autocomplete="off">

```

Save. Nothing else changes in the HTML, and the CSS does not change.

8b. Rewrite `public\js\app.js`

The Day 2 `app.js` kept the notes in an array. The new one asks the server. Many parts are the same as Day 2.

1. Open `public\js\app.js`.
2. Press **Ctrl + A** to select everything, and press **Delete**. The file is now empty.
3. Type the code below, **one section at a time**. Your instructor will explain each section before you type it.

The first lines and Section 1: Find the parts of the page

File: `notes-board\public\js\app.js`

```
// app.js (Day 3)
//
// The Notes Board now talks to the server.
// The notes are kept on the server, not in this file.
// We use fetch() to ask the server for notes, to add a note and to delete one.
// The notes now stay when you refresh the page.
// (They are still lost when the server stops. Day 4 fixes that.)

// 1. Find the parts of the page we need to change.
const form = document.getElementById("note-form");
const titleInput = document.getElementById("note-title");
const textInput = document.getElementById("note-text");
const passwordInput = document.getElementById("note-password");
const charCount = document.getElementById("char-count");
const message = document.getElementById("form-message");
const notesList = document.getElementById("notes-list");
const noteCount = document.getElementById("note-count");
```

Sections 2 and 3: Show a message, and draw the notes

Add this below the code above:

```
// 2. Show a short message under the form.
function showMessage(text, isError) {
  message.textContent = text;
  message.className = isError ? "message error" : "message ok";
}

// 3. Draw a list of notes on the page.
function renderNotes(notes) {
  notesList.textContent = "";

  if (notes.length === 0) {
    const empty = document.createElement("p");
    empty.className = "empty";
    empty.textContent = "No notes yet. Add your first note above.";
    notesList.appendChild(empty);
  }

  for (const note of notes) {
    const article = document.createElement("article");
    article.className = "note";

    const heading = document.createElement("h3");
    heading.textContent = note.title;

    const body = document.createElement("p");
    body.textContent = note.text;

    const deleteButton = document.createElement("button");
    deleteButton.type = "button";
    deleteButton.className = "delete-button";
    deleteButton.textContent = "Delete";
    deleteButton.addEventListener("click", function () {
      deleteNote(note.id);
    });

    article.appendChild(heading);
    article.appendChild(body);
    article.appendChild(deleteButton);
    notesList.appendChild(article);
  }
}
```

```

    noteCount.textContent = notes.length;
  }

```

Note the difference from Day 2: `renderNotes(notes)` now **receives** the list. It does not keep it.

Section 4: Ask the server for the notes (GET)

```

// 4. Ask the server for all notes, then draw them.
async function loadNotes() {
  try {
    const response = await fetch("/api/notes");
    const notes = await response.json();
    renderNotes(notes);
  } catch (error) {
    showMessage("Could not reach the server. Is it running?", true);
  }
}

```

Section 5: Send a new note (POST)

```

// 5. Send a new note to the server.
async function addNote(title, text) {
  const response = await fetch("/api/notes", {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      "x-notes-password": passwordInput.value
    },
    body: JSON.stringify({ title: title, text: text })
  });

  if (!response.ok) {
    const problem = await response.json();
    throw new Error(problem.error);
  }
}

```

Section 6: Ask the server to delete a note (DELETE)

```
// 6. Ask the server to delete a note.
async function deleteNote(id) {
  try {
    const response = await fetch("/api/notes/" + id, {
      method: "DELETE",
      headers: { "x-notes-password": passwordInput.value }
    });

    if (!response.ok) {
      const problem = await response.json();
      throw new Error(problem.error);
    }

    showMessage("Note deleted.", false);
    loadNotes();
  } catch (error) {
    showMessage(error.message, true);
  }
}
```

Sections 7, 8 and 9: The form, the counter and the first load

```
// 7. When the form is submitted, check the input and send the note.
form.addEventListener("submit", async function (event) {
  event.preventDefault();

  const title = titleInput.value.trim();
  const text = textInput.value.trim();

  if (title === "" || text === "") {
    showMessage("Please write both a title and a note.", true);
    return;
  }

  try {
    await addNote(title, text);
    titleInput.value = "";
    textInput.value = "";
    charCount.textContent = "0";
    showMessage("Note added.", false);
    titleInput.focus();
    loadNotes();
  } catch (error) {
    showMessage(error.message, true);
  }
});

// 8. Show how many characters are used while the user types.
textInput.addEventListener("input", function () {
  charCount.textContent = textInput.value.length;
});

// 9. Load the notes once when the page opens.
loadNotes();
```

4. Save the file. You do **not** need to restart the server, because we changed only files in `public`.
5. In the browser, open `http://localhost:3000` and press **Ctrl + F5**. This reloads the page and also reloads its JavaScript.

Check: The page shows a **Password** box. The notes you see are the notes from the server. The Console (F12) shows **no red errors**.

Part 9: Test everything (8 minutes)

First, **restart the server** (**Ctrl + C**, then `node server.js`), so the notes are back to the two starting notes. Refresh the page.

Do each test and tick it.

Add with the right password. Type a title, a note and your password from `.env`. Click **Add note**. The note appears, with the message `Note added.`

Add with a wrong password. The message `Wrong password.` appears in red, and no note is added. Try it also with an empty password box.

Empty title. The message `Please write both a title and a note.` appears.

Refresh keeps the notes. Press **F5**. Your note is **still there**. On Day 2 it disappeared. Now it lives on the server.

Delete. Click **Delete** with a wrong password: `Wrong password.` Type the right password and click **Delete**: `Note deleted.`

Safe text. Add a note with the text `<b>bold</b>`. It shows as plain letters. This is `textContent` working.

A second tab. Open `http://localhost:3000` in a second browser tab. Add a note in the first tab, then press **F5** in the second tab. You see the same notes. The notes belong to the server, not to one tab.

Public reading. Open `http://localhost:3000/api/notes`. You can read the notes without any password.

Restart the server. Press **Ctrl + C** in the terminal and run `node server.js` again. Refresh the page.

Check: Look at the last test. The notes you added are **gone**. Only the two starting notes are back.

Why? `data.js` keeps the notes in the **memory** of the server program. When the server stops, the memory is cleared. This is **not a bug**. It is the lesson for Day 4: we will keep the notes in a **database** on the disk.

Optional, if you have time: stop the server, and try to add a note in the page that is still open. The page cannot reach the server, so you see an error message. (The exact words depend on the browser. Not tested.)

Part 10: Backup, quiz and homework (5 minutes)

1. Copy your `Workshop` folder to a USB drive or to the place your instructor tells you. You may leave out the `node_modules` folder, because `npm install` makes it again. Your `.env` file holds your password, so keep your backup private.

2. Do the 5-question quiz that your instructor shares.
 3. Read the **Homework for Day 4** at the end of the Day 3 Theory.
-

Final checklist

`npm ls express` shows Express, and my `package.json` has `express` in `dependencies`.

My server starts with `node server.js` and my page opens at `http://localhost:3000`.

`GET /api/notes` shows my notes as JSON.

The password is in `.env`, `.env` is listed in `.gitignore`, and the password is not in any `.js` file.

Adding and deleting work with the right password, and show `Wrong password.` with a wrong one.

My notes stay after a refresh, and I can explain why they are lost when the server restarts.

I made a backup copy of my `Workshop` folder.

Extra challenges (only if you finish early)

1. Change `PORT` in `.env` to `3001`, restart the server, and open `http://localhost:3001`.
2. Add a new route `GET /api/hello` in `server.js` that answers `{ "message": "Hello" }` with `response.json(...)`. Open it in the browser.
3. In the Console of the Notes Board page, run `fetch("/api/notes").then(r => r.json()).then(console.log)`. What do you see?
4. Change the limit of the note from 300 letters to a different number. You must change it in **two** places: `index.html` and `server.js`. Why both?
5. Show the time each note was created. Hint: save `new Date().toISOString()` in `addNote` in `data.js`, and show it in `renderNotes`.